

SESSION - 05

Content Rendering



Table of Contents

Introduction To Content Rendering

Exercise – Create the project Structure in /apps ..... 6.

Exercise – Create a Page rendering component ..... 9.

Exercise – Create Content to be rendered ..... 11.

Exercise – Basic Sling resource resolution ..... 17.

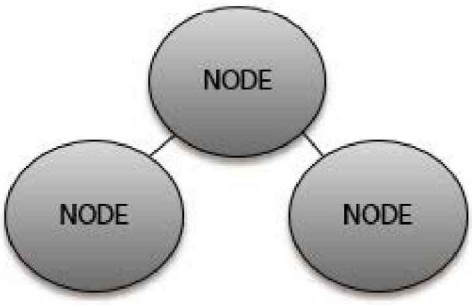
Exercise – Selector Manipulation ..... 19.

INTRODUCTION TO CONTENT RENDERING

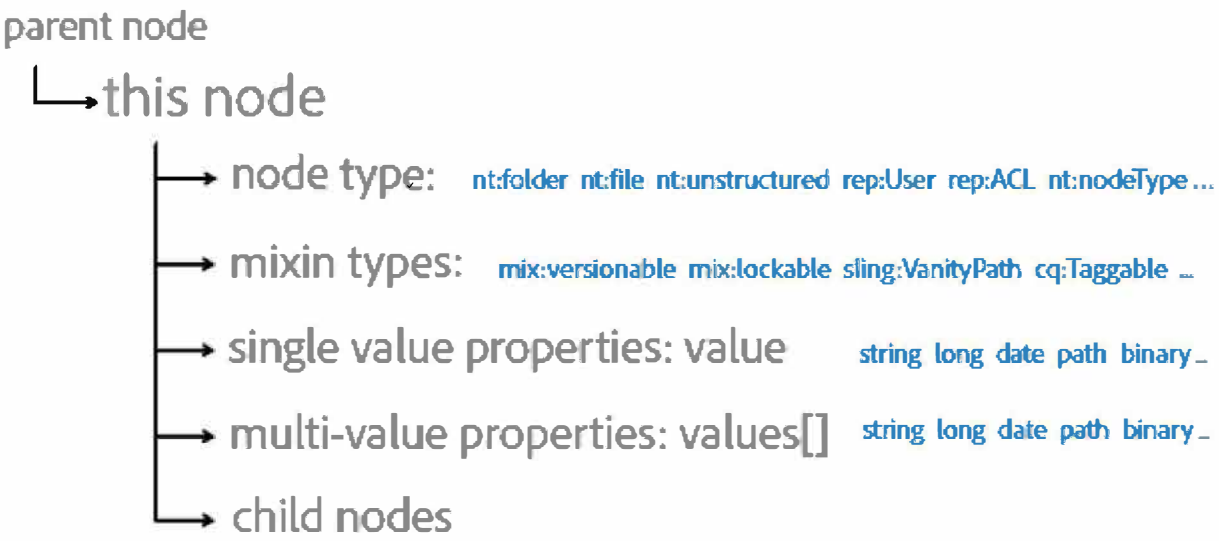
Nodes and Properties

You can think of the JCR as a hierarchical tree of two types of items:

1. Nodes: provide the structure. Nodes can have parent and child nodes which are represented with paths.
2. Properties: store the data. All properties have a name and a value.







Looking more closely at a node (example above), there are a number of other features worth noting:

- Nodes have a type. The node type sets rules about what kinds of properties and child nodes a node can or must have. For example, a node of type cq:page must always have a child node named jcr:content of type cq:PageContent.

- Nodes can also have zero to many mixin types. A mix-in type is an add-on; it specifies additional properties and child nodes the node must or may have. Common mixin types include mix:versionable and mix:referenceable.

Looking at the properties, a node may have any number of properties for storing information. Nodes can be single or multi-valued. A multi-valued property is notated and behaves much like an array. Multi-valued properties usually have plural names. So, here (above) we see a single value property named value and a multi-valued property named values[ ].

Namespaces

Common Namespaces for node types and properties:

- jcr: basic data storage (part of jcr spec)
- nt: foundation node types (part of jcr spec)
- rep: repository internals (part of jcr spec)
- mix: standard mixin node types (part of jcr spec)
- sling: added by Sling framework
- cq: added by the AEM application

Common JCR Node Types

Node Type	Description
nt:file	Represents a file, as in a filesystem.
nt:folder	Represents a folder, as in a filesystem
nt:unstructured	• Allows any combination of child nodes • Allows any combination of properties • Supports client-orderable child nodes • Used to store unstructured content

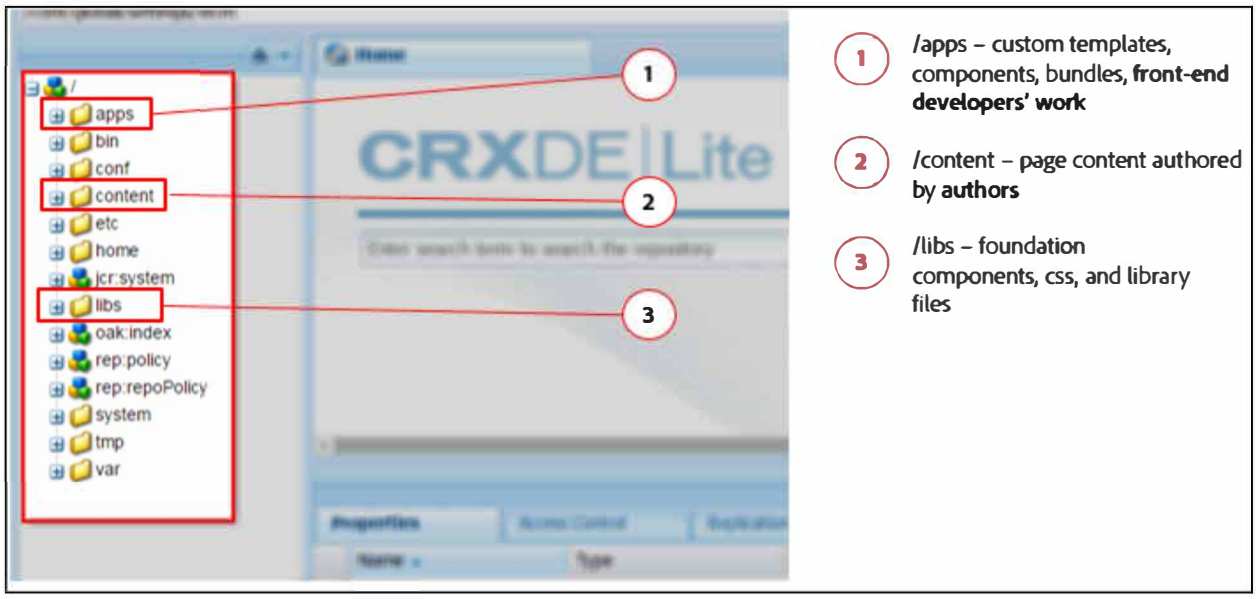
Common AEM Node Types

Node Type	Description
cq:Page	Stores content and properties for a page in a website
cq:Template	Defines a template used to create pages
cq:ClientLibraryFolder	Defines a library of client-side JavaScript CSS
cq:Dialog	Defines a touch-enabled Dialog box
cq:EditConfig	Defines editing configuration for a component including drag and drop and in-place editing.
cq:InplaceEditingConfig	Defines an in-place editing configuration for a component. Always seen as a child of cq:EditConfig

Folder Structure of the Repository

You have already seen how a project created using the Archetype 10 is structured. We will create a similar structure for our site. It is good practice to set up a specific folder structure that clearly defines and organizes the various elements of your site into its respective folders. These elements include your templates, components, OSGi bundles, and static files. You should create all custom projects under the /apps folder in CRXDE Lite.

Before we begin building our site, let's have a quick look at the folder structure in CRXDE Lite.



Here's what each of the folders contain:

/apps—stores all custom templates, components, and any other definitions related to your site are stored here. When you inherit foundation components, it is done from the libs folder. Always copy the components from the libs folder to the apps folder, and then modify it in the apps folder. By doing this, no code will be lost when you upgrade Adobe Experience Manager. You just need to update the apps folder.

NOTE: A best practice is to use a feature known as Sling Resource Merger (which will be discussed in **future** sessions – Advanced Sling Functionality), and just duplicate the required /libs folder structure (empty folders) under /apps. You can then make modifications to the node or property, or wherever the changes are required.

/libs— stores all libraries and definitions that belong to Adobe Experience Manager code are stored here. It includes out-of-the-box components, templates, and other Adobe Experience Manager features such as search or replication. It is also referred to as the foundation components. **Avoid making any modifications to any of the components in libs**.

/content—all content of your website is stored in this folder.

/conf—contains all configurations for your site. This folder is used to store the dynamic templates and policies for your site.

/etc—contains all resources related to utilities and tools.

/home—contains all information related to users and groups.

/tmp—serves as a temporary working area.

/var—contains files that change and are updated by the system, such as audit logs, statistics, and event handling.

/oak:index—is a node that contains Jackrabbit Oak index definitions. Each node specifies the details of one index. Standard indexes for the AEM application are visible and additional custom indexes can be created.

Exercise - I

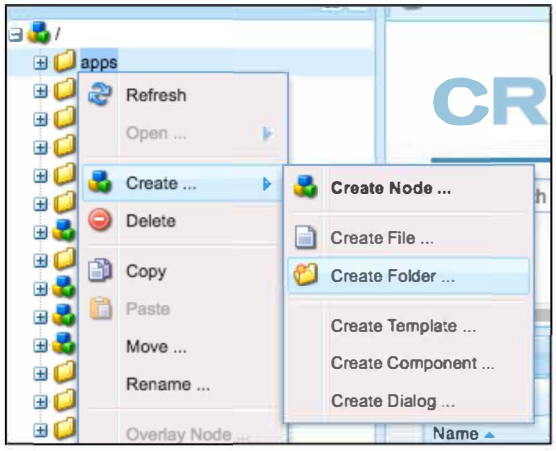
In one of the earlier modules, we touched on Apache Sling, the web framework. Now let's go a bit deeper and see how AEM uses Apache Sling to resolve a resource and then map that resource to a rendering script.

Task – Create the project structure in /apps

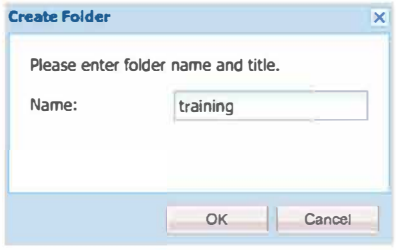
1. **Navigate to CRXDE Lite. http://localhost:4502/crx/de. Or from any console, enter "crx" into the OmniSearch toolbar and click on CRXDE Lite from the search results. You may load the OmniSearch at any time by typing "/" in AEM.**



2. **Right-click on /apps and select Create...> Create Folder.**



3. **Enter training as the name of the folder. Click OK and then click Save All in the upper-left.**



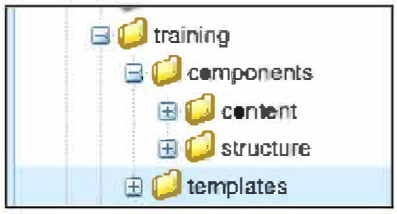
NOTE: Node names are a restricted set. No spaces or special characters. Also, do not use "_" (underscore). By convention, node names begin with a lowercase letter.

4. **Right-click on /apps/training and select Create...> Create Folder. Enter "components" as the folder name.**
5. **Save your changes using Save All. From this point forward in this course, you will click Save All to save any and all changes in CRXDE Lite. It is a best practice to click Save All often!**

6. **Now use what you learned to create two child folders of /apps/training/components named "content" and "structure". Save your changes.**



7. **To finish creating the project structure, right-click on /apps/training and create a folder named templates. Save your changes.**



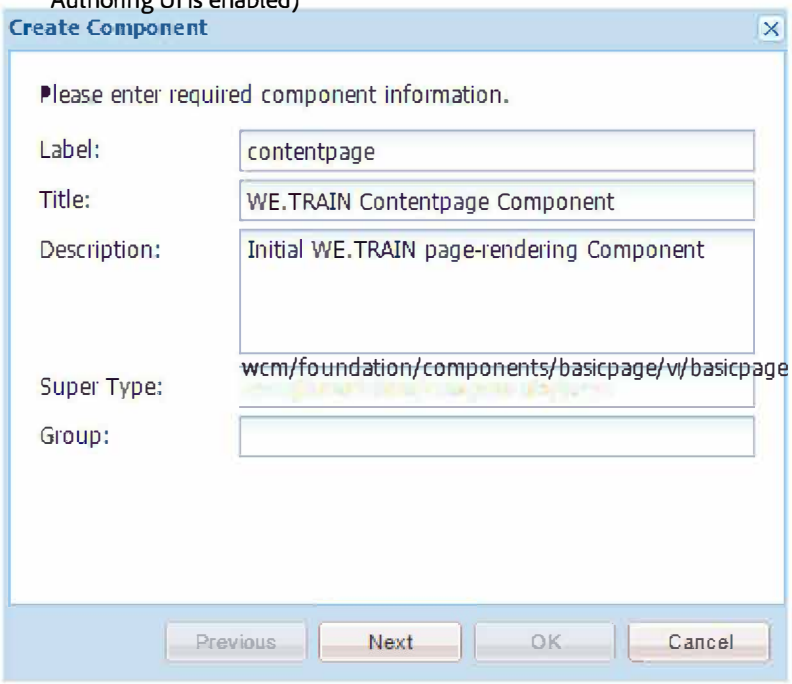
Page Rendering Components

- Components are modular, reusable units that implement specific functionality or logic to render the content of your website.
- A component is a collection of scripts (for example, HTL files, JSPs, Java servlets, and so on) that completely realize a specific function.
- Every component has a default script file identical to the name of the component. If needed, you can remove this script and create your own.
- The template sling:resourceType property must match the path to a page rendering component.

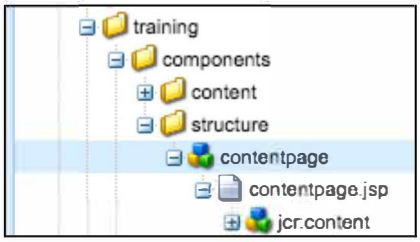
Exercise – II

Task – Create a page-rendering component

1. **Right-click /apps/training/components/structure and select Create...> Create Component.**
2. **Fill in the following values:**
   - **Label**: contentpage
   - **Title**: WE.TRAIN Contentpage Component
   - **Description**: Initial WE.TRAIN page-rendering Component
   - **SuperType**: wcm/foundation/components/basicpage/v1/basicpage (This indicates the Authoring UI is enabled)



3. **Click Next.**
4. **Click OK, and save your changes. Your folder structure should look like this:**



5. **Right-click on contentpage.jsp to rename it to contentpage.html. You are renaming this because you are populating this with HTML.**
6. **Save All.**
7. **Double-click on contentpage.html to open the script for editing.**
8. **Replace all sample code and comments with the HTML code from the Exercise Files for this course. The file is provided in the exercise folder.**

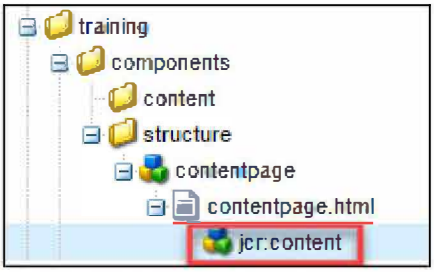


NOTE: There are two ways you can replace HTML, JS, and any other code in CRXDE Lite in this entire course (OPTION A and OPTION B):

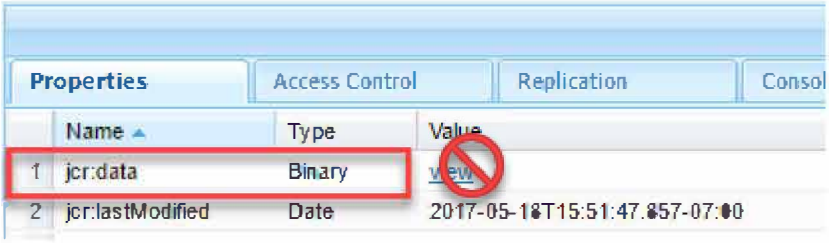
OPTION A: Open the code files from the Exercise folder in Notepad++ (or similar text editor), copy all the contents of the files, and paste the contents into CRXDE Lite. Save All!

OPTION B: Use CRXDE Lite's method of populating code in the jcr:data node. The steps are as follows. You may follow these steps as-is to complete Step 8:

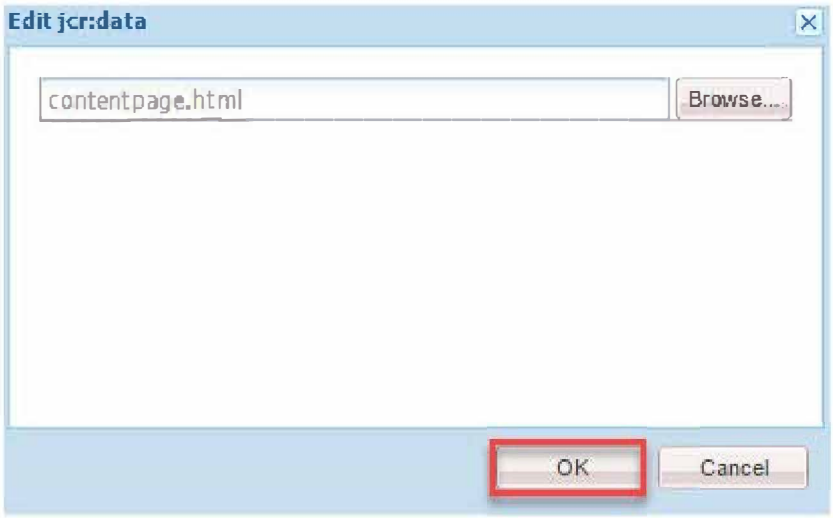
a. Under the renamed (or newly created) file (in this case, **contentpage.html**), find a **jcr:content** node. Expand this node:



b. **Double-click on the jcr:data property within this node. Do NOT click on the View link:**



c. **In the edit dialog that appears, click Browse and select the file in the Exercise directory on your computer components/contentpage/contentpage.html):**



d. **Click Ok. Save All in CRXDE Lite.**

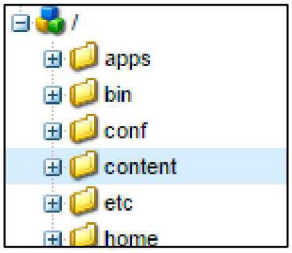
NOTE: You may use this method to replace the contents of files as well. You can use this method for all situations where you create a new file, or replace the contents of files. If you are creating a new blank file, remember to first save the file after it is initially created. Then use the method of double-clicking on the jcr:data property.

Task – Create content to be rendered

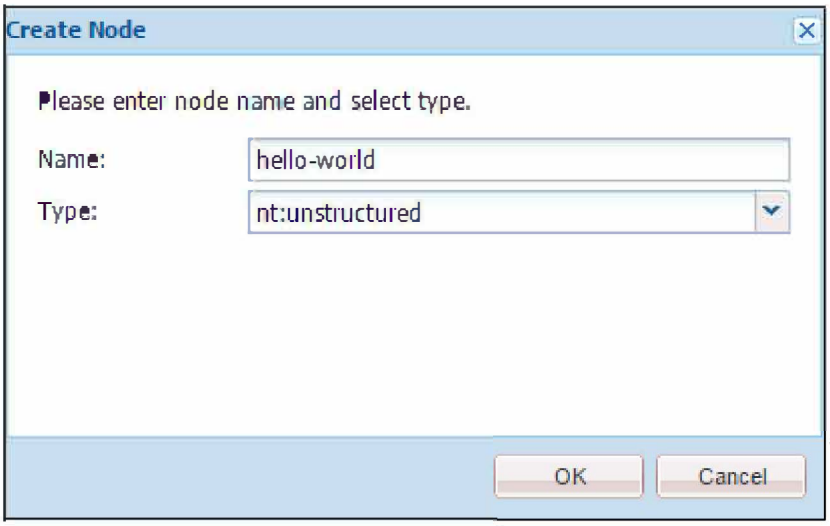
Now you have a component that will render content, we need some actual content to render.

1. **Navigate in CRXDE Lite to /content.**

   NOTE: This is the at the root, not within your **/training** project structure.



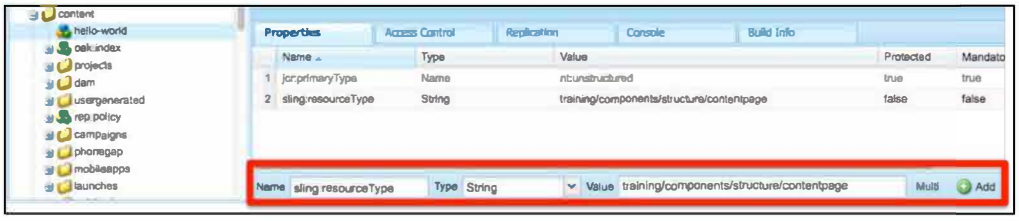
2. **Right-click on /content and select Create…> Create Node.**
3. **Create a node named "hello-world" of type "nt:unstructured":**



4. **Save your changes.**
5. **In your node, add a property with the following attributes, and click Add.**

Name	Type	Value
sling:resourceType	String	training/components/structure/contentpage

Notice the sling:resourceType property value is the path to the page-rendering component that we just defined.



Pro Tip: By convention, property names are camel case, excluding the namespace prefix (sling:). Camel case means the first word in the property name starts with a lowercase letter and each additional word in the property name starts with an uppercase letter (in the above, **resourceType** follows the camel case syntax).

6. **Save your changes.**

Open http://localhost:4502/content/hello-world.html in the browser.



7. What was the sequence of events that caused the Hello World text to display?

Understanding the Sling Resolution Process

- Sling is resource-oriented.
- All resources are maintained in the form of a virtual tree.
- A resource is usually mapped to a JCR node, but can also be mapped to a file system or database.
- Common properties that a resource can have:
    - Path
    - Name
    - Resource Type

URL ➔ Resource = JCR Structure

Basic Steps of Processing Requests

Each content item in the JCR repository is exposed as an HTTP resource.

After the content is determined, the script or servlet to be used to handle the request is determined through:

- Properties of the content item itself
- The HTTP method used to make the request
- A simple naming convention within the URL that provides secondary information

For every URL request, the following steps are performed to get it resolved:

1. Decompose the URL
2. Search for a servlet or a vanity URL redirect
3. Search for a node indicated by the URL
4. Resolve the Resource
5. Resolve the rendering script/servlet
6. Create rendering chain
7. Invoke rendering chain

Decomposing the URL

Consider the following example:

URL: http://myhost/tools/spy.printable.a4.html/a/b?x=12

The above URL can be decomposed into the following components:

protocol	Host	Content path	Selector(s)	Extension		Suffix		Param(s)
http://	myhost	tools/spy	.printable.a4	html	/	a/b	?	x=12

where:

- **Protocol:** Hypertext transfer protocol (HTTP)
- **Host:** Name of the website
- **Content path:** Path specifying the content to be rendered.

- **Selector(s):** Used for alternative methods of rendering the content
- **Extension:** Content format; also specifies the script to be used for rendering.
- **Suffix:** Can be used to specify additional information
- **Param(s):** Any parameters required for dynamic content

Resolving Requests to Resources

Let's take a look at a couple of examples:

Example 1:

```
URL: http://myhost/tools/spy.html
```

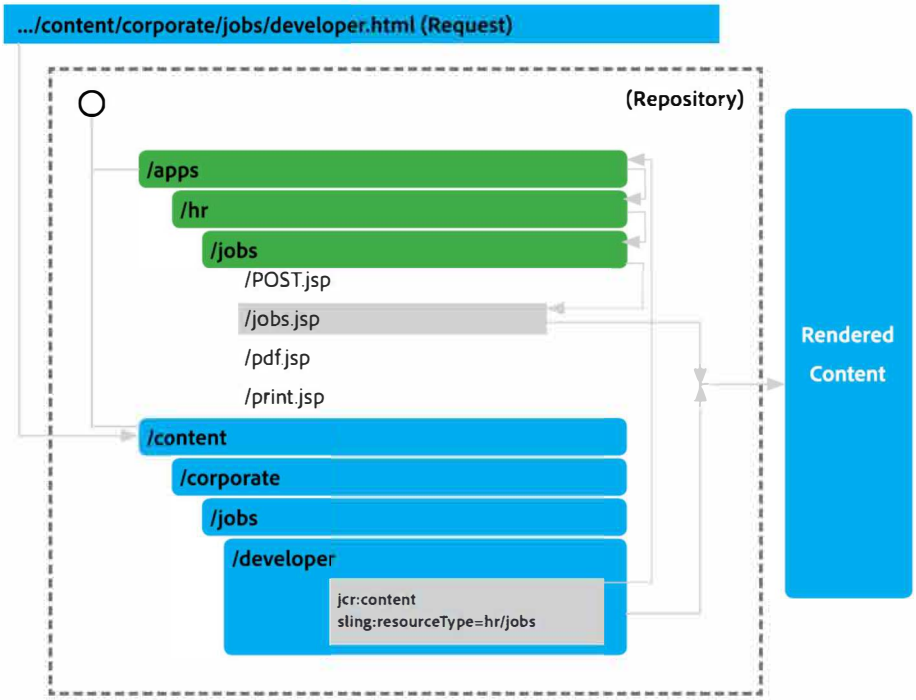
Once the URL is decomposed, the content node is located from the content path. This node is identified as the resource, and performs the following steps to map to the request.

Steps performed to map request:

1. The Sling Resource Resolver process first looks for a redirection rule (such as a vanity URL), or a servlet and resolves to this. If this does not exist or is not found, then the script resolution process begins.
2. As part of the script resolution process, Sling searches for the **spy** node.
3. If a node is found, then the sling resource type for that node is extracted, and used to locate the script to be used for rendering the content.
4. If no node is found, then Sling will return the http code 404 (Not Found).

Example 2:

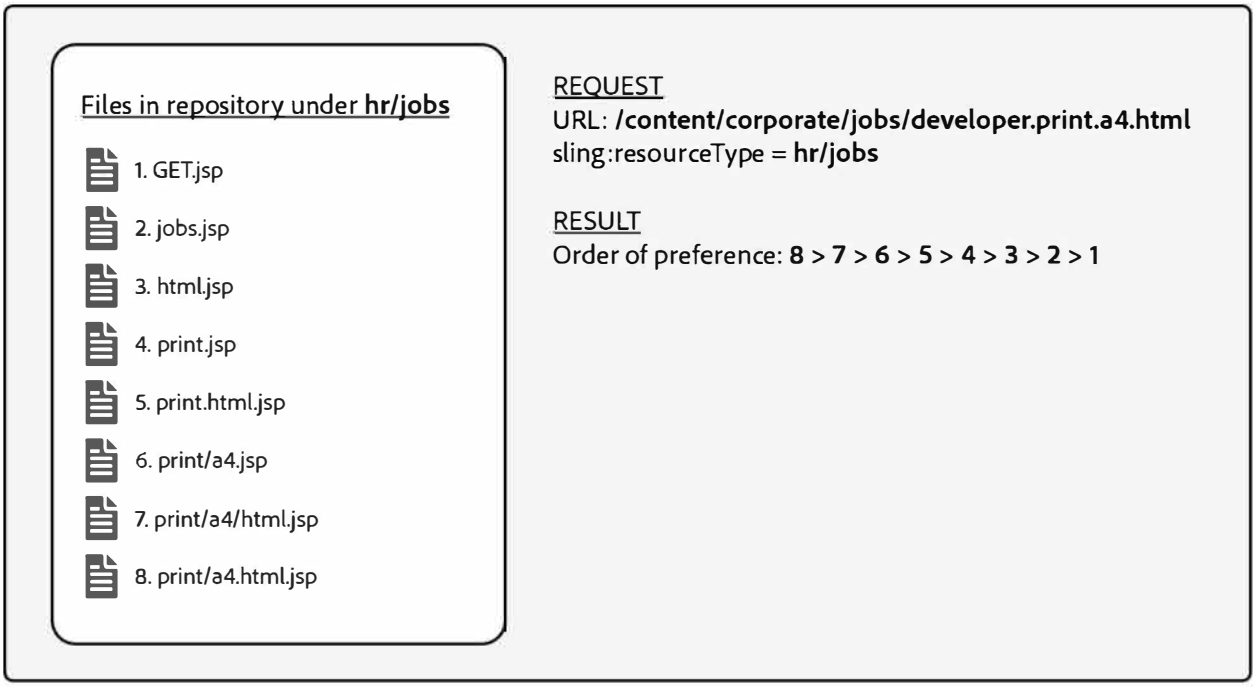
Consider the URL request: http://myhost/content/corporate/jobs/developer.html and corresponding diagram. Notice the properties of the developer node are used to provide the rendered content:



Locating and Rendering Scripts

When the resource is identified from the URL, its resource type property is located and the value extracted. This value is either an absolute or a relative path that points to the location of the script to be used for rendering the content. All scripts are stored in either the /apps or /libs folder, and are searched in the same order. If no matching script is found in either of the folders, then the default script is rendered.

For multiple matches of the script, the script name with the best match is selected. The more selector matches, the better.

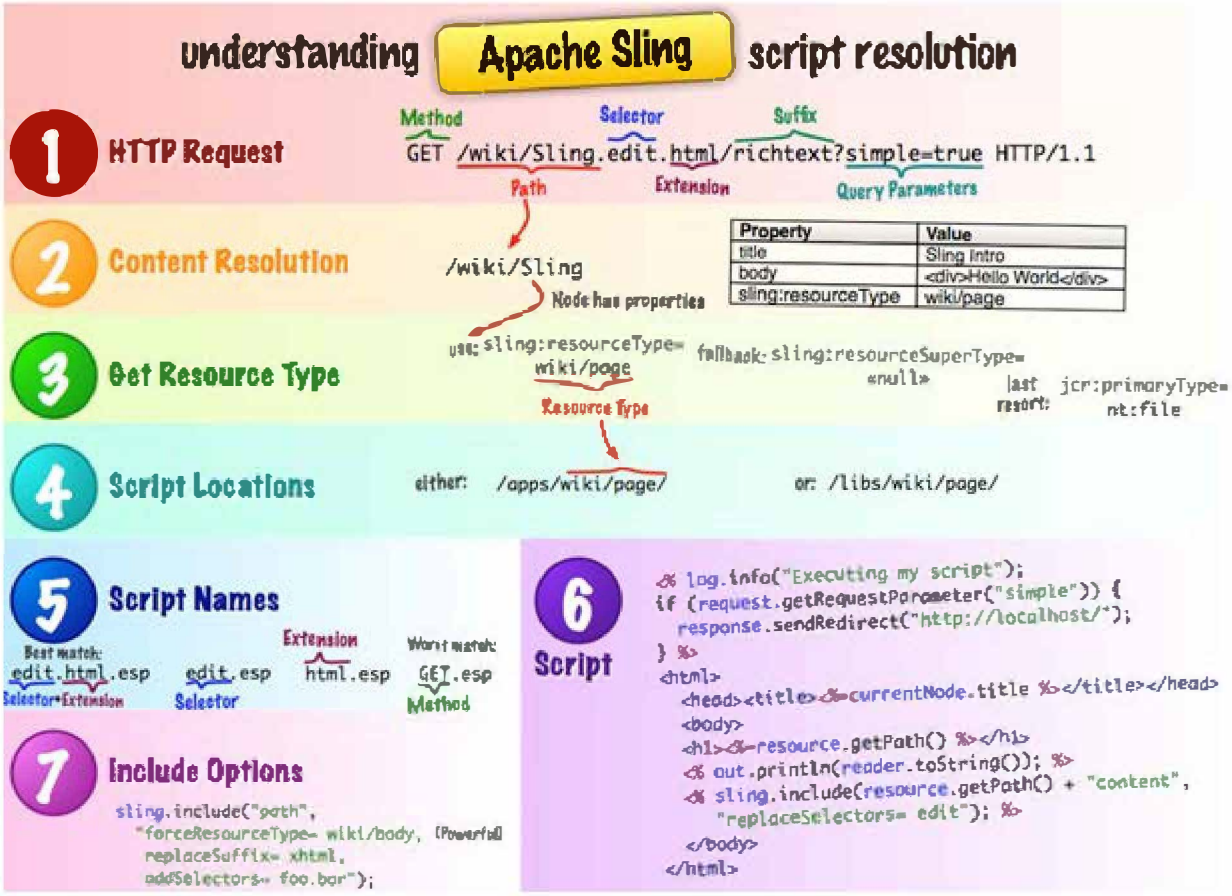


Understanding URL Decomposition

Each content item in the JCR repository is exposed as an HTTP resource, so the request URL addresses the data to be processed, not the procedure that does the processing. After the content is determined, the script or servlet to be used to handle the request is determined in cascading resolution that checks, in order of priority:

1. Properties of the content item itself
2. The HTTP method used to make the request
3. A simple naming convention within the URL that provides secondary information

Therefore, it must be noted again that chronologically, the **Sling Resource Resolver first** tries to resolve the URL to a servlet or redirect. If this does not exist or is not successful, then the **script resolution process** described below takes place:



If no node is found at all, a 404 error is returned.

Exercise – III

Task – Basic Sling resource resolution and hunt for a rendering script

When presented with a request (URL), Apache Sling will take the following actions:

1. **Strip off the protocol, server, and port.**
2. **Strip off any suffix.**
3. **Examine the remaining portion of the URL and use the information contained therein to assist with resolving the resource and finding its associated rendering script.**

Let us follow Sling's actions to see how our hello-world resource was rendered.

1. **Consider the request (URL) http://localhost:4502/content/hello-world.html**
2. **Disregard http://localhost:4502.**
3. **In CRXDE Lite, navigate to /content/hello-world.html. Notice that path does not exist in the repository.**



4. **At this point, Sling will back off the end of the remaining part of the request until the first ".". Everything after the "." is considered the extension. Now Sling will attempt to locate "/content/hello-world".**



5. **In CRXDE Lite, navigate to /content/hello-world. Yay! We found a node in the repository that matches the resource in the request. What we just did is called "resolving the resource". We successfully resolved a request URL to a resource that is represented by a node in the repository.**



NOTE: Resources could also map to items in relational databases and/or file system. The Apache Sling specification does not mandate a JCR database.

6. **Now we need to find the rendering script. On the /content/hello-world node, find the sling:resourceType property. The value of the sling:resourceType property is what Sling uses to begin its search for a rendering script.**
7. **Notice the sling:resourceType property points to the page-rendering script that we created previously.**



8. **Navigate to: /apps/training/components/structure/contentpage**
9. **Notice there is one script: contentpage.html. This is called the default script because the script's name matches the name of the folder/component in which the script resides. Later, we will explore the many options that Sling might use in selecting the "right" script. But right now, we have the default script. Given the specified request and the available selection of scripts, the default script is the best match and Sling chooses it to render the resource.**

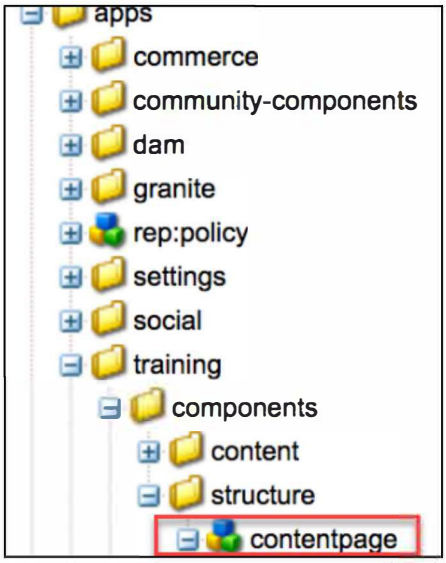


10. **The script outputs "Hello World!". We have now successfully resolved the resource, found the rendering script, and invoked the rendering chain.**
11. **You have just seen how the request "http://localhost:4502/content/hello-world.html" results in the following browser output:**



Task – Selector Manipulation

1. **Using CRXDE Lite, right-click on /apps/training/components/structure/contentpage.**



2. **Select Create… > Create File.**
3. **Enter blue.html as the file name.**
4. **Click OK. Save your changes.**
5. **Open the file blue.html and enter the following statement. You will find the code in the Exercise Files as well.**

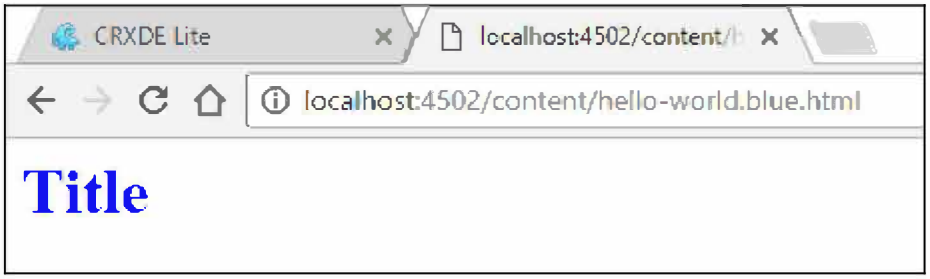
   <h1 style="color:blue">Title</h1>

6. **Save your changes.**

   To test our selectors, enter the URL in your browser that will cause our hello-world node to be rendered: **http://localhost:4502/content/hello-world.html.**



7. **Now enter http://localhost:4502/content/hello-world.blue.html. Notice the difference. The code from blue.html is chosen, as the script name matches the selector in the URL.**



We will continue to learn more about Apache Sling URL decomposition and resource resolution throughout the course.